



Storing Measurements: Isolation vs. Simplicity

Why are we talking about this?

Last Monday, you recommended a Design-Approach for our Time-Series-Database...

... a Patient has an own seperated Bucket which stores all Measurements of this Patient

We are talking about this because this will cause a lot of problems

Why is that a Problem? The Problem is the Bucket itself?

A Bucket is a heavyweight logical unit, has it's own retention policy, internal metadata, ...

Influx is optimized for few buckets, with many series inside each bucket

Cross-Patient queries become very slow and hard to manage

Per-Patient Isolation looks nice, but does neither scale, nor be simple to manage and maintain

Why is this a Problem? Writing to our Database

Telegraf does not dynamically choose buckets per metric

Theoretical Workarounds:

1. One Telegraf instance Per-Bucket/Patient (only feasible for very small N)
2. Single Telegraf with multiple Outputs + Metric Filtering (reconfigure Telegraf for new Patients)
3. Don't use Telegraf (Reimplement: Rounting, Auth, Retry Logic, Buffering, Backpressure, ...)

The Uncomfortable but Honest Recommendation

If you want: Scalability, Simplicity, and Reliability
=> One Bucket + Patient-ID as tag

If you accept complexity, accept operational overhead, and must need hard storage isolation
=> Per-Patient Bucket + Custom Write Service

But does that mean we give up, all Isolation?

Actually not at all

We isolation on the Application Layer and the Authentication Layer:

- Telegraf has an **Scoped Write Token**
- Grafana has only an **Scoped Read Token** (Doctors must Authenticate)
- Sensitive User Data is stored in separate/independent Relational Database